```
    public void OnResourceExecuting(          ❶
        ResourceExecutingContext context)     ❷
    {
        Console.WriteLine("Executing!");
    }

    public void OnResourceExecuted(           ❸
        ResourceExecutedContext context)      ❹
    {
        Console.WriteLine("Executed");
    }
}
```

❶ Executed at the start of the pipeline, after authorization filters

❷ The context contains the HttpContext, routing details, and information about the current action.

❸ Executed after model binding, action execution, and result execution

❹ Contains additional context information, such as the IActionResult returned by the action

The interface methods are simple and are similar for each stage in the filter pipeline, passing a context object as a method parameter. Each of the two-method sync filters has an `*Executing` and an `*Executed` method. The type of the argument is different for each filter, but it contains all the details for the filter pipeline.

For example, the `ResourceExecutingContext` passed to the resource filter contains the `HttpContext` object itself, details about the route that selected this action, details about the action itself, and so on. Contexts for later filters contain additional details, such as the action method arguments for an action filter and the `ModelState`.

The context object for the `ResourceExecutedContext` method is similar, but it also contains details about how the rest of the pipeline executed. You can check whether an unhandled exception occurred, you can see if another filter from the same stage short-circuited the pipeline, or you can see the `IActionResult` used to generate the response.